

Design Assignment 6

Student Name: Joshua Martinez

Student #: 5007531628

Student Email: marti87@unlv.nevada.edu

Primary Github address: <https://github.com/JoshuaMa2003>

Directory: https://github.com/JoshuaMa2003/submission_da

Video Playlist:

https://www.youtube.com/playlist?list=PLZ09MA_uFt63me60r5XWtDUj2m21n4nUP

PLEASE NOTE: AS OF SUBMITTING THIS ASSIGNMENT ON 5/4/2024, MY ATMEGA328PB MICROCONTROLLER WAS NOT FUNCTIONING, SO I WAS NOT ABLE TO GET ANY VIDEOS OR PICTURES OF MY BOARD

1. COMPONENTS LIST AND DIAGRAM SHOWING PINS USED

List of Components used

- **Microchip Studio Debugger**
- **Microchip Studio Simulator**
- **ATmega328PB Microcontroller**
- **DC Motor**
- **ROB-14450 Motor Driver**
- **ATmega328PB Microcontroller Potentiometer**
- **Male-to-Male Wires**
- **Multifunctional Shield Seven Segment Display**

2. INITIAL CODE OF TASK

```
#define F_CPU 16000000UL          //define the clock frequency as 16 mhz
#include <avr/io.h>                //include the input/output library for avr
#define ENABLE 3                  //define enable pin as pb3
#define MTR_1 1                   //define motor 1 pin as pb1
#define MTR_2 2                   //define motor 2 pin as pb2
#define SW (PIND & (1<<7))       //read switch state from pd7
#include <util/delay.h>            //include the delay functions
#include <stdio.h>                 //include the standard input/output library
#include <avr/interrupt.h>         //include the interrupt library
```

```
//function declarations
```

```
void read_adc(void);
```

```
void adc_init(void);
```

```
volatile unsigned int adc_temp;   //variable to store adc result
```

```
//initialize adc settings
```

```
void adc_init(void)
```

```
{
    ADMUX = (0<<REFS1)|           //use avcc as reference voltage
    (1<<REFS0)|
    (0<<ADLAR)|                   //right adjust adc result
    (0<<MUX2)|                     //use adc0 (pc0, pin23)
    (0<<MUX1)|
    (0<<MUX0);
    ADCSRA = (1<<ADEN)|           //enable adc
    (0<<ADSC)|                     //do not start conversion
    (0<<ADATE)|                   //disable auto trigger
    (0<<ADIF)|                     //clear adc interrupt flag
    (0<<ADIE)|                     //disable adc interrupt
    (1<<ADPS2)|                   //set adc prescaler to 128
    (0<<ADPS1)|
    (1<<ADPS0);
}
```

```
//read adc value
```

```
void read_adc(void)
```

```
{
    unsigned char i = 4;
    adc_temp = 0;
    while (i-->0)
    {
```

```

        ADCSRA |= (1<<ADSC);    //start conversion
        while (ADCSRA & (1<<ADSC)); //wait for conversion to finish
        adc_temp += ADC;        //sum up results
        _delay_ms(50);          //wait 50 ms
    }
    adc_temp = adc_temp / 4;      //average four samples
    adc_temp = adc_temp * 50 / 1024; //convert to voltage scale
}

int main(void)
{
    adc_init();                  //initialize adc
    PORTD |= (1<<7);             //enable pull-up resistor on pd7
    DDRB |= 0b00001110;         //set pb3, pb1, pb2 as outputs
    PORTB &= ~(1<<ENABLE);       //disable motor initially
    PORTB &= ~(1<<MTR_1);        //turn off mtr_1
    PORTB &= ~(1<<MTR_2);        //turn off mtr_2
    DDRB |= (1<<3);              //set oc2a as output
    OCR2A = 50;                  //set output compare for pwm

    //configure timer 2 for fast pwm
    TCCR2A = (1<<COM2A1)|(1<<WGM21)|(1<<WGM20);
    TCCR2B = 0x02;               //set prescaler to 8

    while (1)
    {
        read_adc();              //read adc value
        PORTD |= (1<<ENABLE);     //enable motor
        if(SW != 0)               //if switch is on
        {
            _delay_ms(20);        //delay for 20ms
            PORTB |= (1<<MTR_1);   //drive mtr_1 high for clockwise rotation
            PORTB &= ~(1<<MTR_2);  //keep mtr_2 low
        }
        else
        {
            _delay_ms(20);        //delay for 20ms
            PORTB &= ~(1<<MTR_1);  //drive mtr_1 low
            PORTB |= (1<<MTR_2);   //drive mtr_2 high for counter-clockwise
rotation
        }
        OCR2A = adc_temp * 15;    //adjust pwm based on adc result
    }
}

```

2. INITIAL CODE OF TASK

```
/*
 * CPE301_DesignAssignment6_Task2.c
 *
 * Created: 5/4/2024 10:46:33 PM
 * Author : joshu
 */

#define F_CPU 16000000UL //define the clock frequency as 16 mhz
#include <avr/io.h> //include the input/output library for avr
#include <util/delay.h> //include the delay functions
#include <stdio.h> //include the standard input/output library
#include <avr/interrupt.h> //include the interrupt library

#define ENABLE 3 //define enable pin as pb3
#define MTR_1 1 //define motor 1 pin as pb1
#define MTR_2 2 //define motor 2 pin as pb2
#define SW (PIND & (1<<7)) //read switch state from pd7

#define BAUDRATE 9600 //define default baud rate for serial communication
#define BAUD_PRESCALER (((F_CPU / (BAUDRATE * 16UL))) - 1) //calculate prescaler for uart

//function declarations
void read_adc(void);
void adc_init(void);
void USART_init(unsigned int ubrr);
void USART_tx_string(char *data);
void InitTimer1(void);
void InitExtInter(void);
void StartTimer1(void);

//global variables for adc, timer counts, etc.
volatile unsigned int adc_temp, revTickAvg, revCtr, ticks, timeTicks, timeCount;
volatile uint32_t tickv, T1Ovs2;
char outs[20];
char please[5] = "1.1\n\0";

//timer 0 overflow interrupt service routine
ISR(TIMERO_OVF_vect) {
    timeTicks++;
    if (timeTicks > 980) {
        sprintf(outs, sizeof(outs), "%.2d\n", ticks / 24); //format ticks to string
        please[0] = outs[0];
        please[2] = outs[1];
    }
}
```

```

        USART_tx_string("RPS: "); //send rpm data via uart
        USART_tx_string(please);
        ticks = 0;
        timeCount = 0;
        timeTicks = 0;
    }
    TCNT0 = 0; //reset timer counter
}

//external interrupt 1 service routine for counting motor revolutions
ISR(INT1_vect) {
    ticks++;
}

//adc initialization
void adc_init(void) {
    ADMUX = (0<<REFS1) | (1<<REFS0) | (0<<ADLAR) | (0<<MUX2) | (0<<MUX1) |
(0<<MUX0);
    ADCSRA = (1<<ADEN) | (0<<ADSC) | (0<<ADATE) | (0<<ADIF) | (0<<ADIE) | (1<<ADPS2)
| (0<<ADPS1) | (1<<ADPS0);
}

//read adc value
void read_adc(void) {
    unsigned char i = 4;
    adc_temp = 0;
    while (i--) {
        ADCSRA |= (1<<ADSC);
        while (ADCSRA & (1<<ADSC));
        adc_temp += ADC;
        _delay_ms(50);
    }
    adc_temp = adc_temp / 4; //average four samples
    adc_temp = adc_temp * 50 / 1024; //convert to voltage
}

//timer 1 initialization
void InitTimer1(void) {
    TCCR0A = 0;
    TCNT0 = 255;
    TIMSK0 = (1 << TOIE0); //enable timer0 overflow interrupt
    TCCR0B |= (1 << CS01) | (1 << CS00); //set prescaler to 64
    sei(); //enable global interrupts
}

```

```

//external interrupt setup
void InitExtInter(void) {
    DDRD &= ~(1 << 2); //set pd2 as input
    PORTD |= (1 << 1) | (1 << 2); //enable pull-ups
    EICRA = (1 << 2); //set int1 to trigger on falling edge
    EIMSK = (1 << INT1); //enable int1
    sei(); //enable global interrupts
}

//usart initialization
void USART_init(unsigned int ubrr) {
    UBRROH = (unsigned char)(ubrr >> 8);
    UBRROL = (unsigned char)ubrr;
    UCSROB = (1 << TXEN0); //enable transmitter
    UCSROC = (3 << UCSZ00); //set frame format: 8 data bits, no parity, 1 stop bit
}

//send string over usart
void USART_tx_string(char *data) {
    while (*data != '\0') {
        while (!(UCSROA & (1 << UDRE0))); //wait for empty transmit buffer
        UDRO = *data++; //put data into buffer, sends the data
    }
}

int main(void) {
    InitExtInter();
    InitTimer1();
    adc_init(); //initialize adc
    USART_init(BAUD_PRESCALLER); //initialize usart
    USART_tx_string("Connected!\r\n"); //send connection message
    _delay_ms(125); //wait a bit

    PORTD |= (1 << 7); //enable pull-up on pd7
    DDRB |= 0b00001110; //set pb3, pb1, pb2 as outputs
    PORTB &= ~(1 << ENABLE); //disable motor initially
    PORTB &= ~(1 << MTR_1); //motor 1 off
    PORTB &= ~(1 << MTR_2); //motor 2 off
    DDRB |= (1 << 3); //set oc2a as output
    OCR2A = 50; //set output compare for pwm
    TCCR2A = (1 << COM2A1) | (1 << WGM21) | (1 << WGM20); //configure for fast pwm
    TCCR2B = 0x02; //set prescaler to 8
}

```

```

while (1) {
    read_adc(); //read adc value continuously
    PORTD |= (1 << ENABLE); //enable motor
    if (SW != 0) { //if switch is on
        _delay_ms(20);
        PORTB |= (1 << MTR_1); //motor 1 on for clockwise rotation
        PORTB &= ~(1 << MTR_2); //motor 2 off
    } else {
        _delay_ms(20);
        PORTB &= ~(1 << MTR_1); //motor 1 off
        PORTB |= (1 << MTR_2); //motor 2 on for counter-clockwise rotation
    }
    OCR2A = adc_temp * 15; //adjust pwm based on adc result
}
}

```


2. INITIAL CODE OF TASK

```
/*
 * CPE301_DesignAssignment6_Task3.c
 *
 * Created: 5/4/2024 10:53:31 PM
 * Author : joshu
 */

#define F_CPU 16000000UL //define clock frequency as 16 MHz for delay calculations
#include <avr/io.h> //include AVR device-specific IO definitions
#include <util/delay.h> //include functions for delay loops
#include <stdio.h> //include standard input and output functions
#include <avr/interrupt.h> //include interrupt handling functionality

#define ENABLE 3 //define enable pin number for motor control
#define MTR_1 1 //define motor 1 control pin
#define MTR_2 2 //define motor 2 control pin
#define SW (PIND << 7) //define switch input using pin PD7
#define SHIFT_REGISTER DDRB //define shift register direction register
#define SHIFT_PORT PORTB //define shift register port
#define DATA (1 << PB3) //define data input pin for SPI (MOSI)
#define LATCH (1 << PB2) //define latch pin for SPI (SS)
#define CLOCK (1 << PB5) //define clock pin for SPI (SCK)

#define BAUDRATE 9600 //define baud rate for USART
#define BAUD_PRESCALLER (((F_CPU / (BAUDRATE * 16UL))) - 1) //calculate USART baud rate prescaler

//function prototypes
void read_adc(void);
void adc_init(void);
void USART_init(unsigned int ubrr);
void USART_tx_string(char *data);
void init_IO(void);
void init_SPI(void);
void spi_send(unsigned char byte);

//global variables
volatile unsigned int adc_temp, revTickAvg, ticks, recTicks, recTicks2, timeTicks, timeCount;
char outs[20];
char please[5] = "1.1\n\0";
const uint8_t SEGMENT_MAP[] = {0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82, 0xF8, 0x80, 0x90}; //segment byte maps for numbers 0-9
const uint8_t SEGMENT_SELECT[] = {0xF1, 0xF2, 0xF4, 0xF8}; //byte maps to select digit 1-4
```

```
volatile int alternator = 0;
```

```
//setup IO for SPI and shift register
```

```
void init_IO(void){  
    SHIFT_REGISTER |= (DATA | LATCH | CLOCK); //set control pins as outputs  
    SHIFT_PORT &= ~(DATA | LATCH | CLOCK); //initialize control pins to low  
}
```

```
//initialize SPI in Master mode
```

```
void init_SPI(void){  
    SPCR0 = (1<<SPE) | (1<<MSTR); //enable SPI and set as Master  
}
```

```
//send byte via SPI
```

```
void spi_send(unsigned char byte){  
    SPDR0 = byte; //load byte to SPI data register  
    while(!(SPSR0 & (1<<SPIF))); //wait until SPI transmission is complete  
}
```

```
//ISR for TIMER0 overflow
```

```
ISR(TIMER0_OVF_vect) {  
    timeTicks++;  
    if(timeTicks > 980){  
        recTicks = ticks;  
        ticks = 0;  
        timeCount = 0;  
        timeTicks = 0;  
    }  
    TCNT0 = 0;  
}
```

```
//ISR for external interrupt 0
```

```
ISR(INT0_vect){  
    ticks++;  
}
```

```
//initialize ADC
```

```
void adc_init(void){  
    ADMUX = (0<<REFS1)|(1<<REFS0)|(0<<ADLAR)|(0<<MUX2)|(0<<MUX1)|(0<<MUX0);  
    //select AVcc as reference, right adjust, select ADC0  
    ADCSRA =  
(1<<ADEN)|(0<<ADSC)|(0<<ADATE)|(0<<ADIF)|(0<<ADIE)|(1<<ADPS2)|(0<<ADPS1)|(1<<ADPS0); //enable ADC, set prescaler to 128  
}
```

```

//read ADC value
void read_adc(void){
    unsigned char i = 4;
    adc_temp = 0;
    while (i--){
        ADCSRA |= (1<<ADSC); //start ADC conversion
        while(ADCSRA & (1<<ADSC)); //wait for conversion to complete
        adc_temp += ADC; //accumulate converted result
        _delay_ms(50);
    }
    adc_temp = adc_temp / 4; //average over four samples
    adc_temp = adc_temp * 50 / 1024; //convert to voltage
}

```

```

//initialize timer0 for periodic interrupts
void InitTimer1(void) {
    TCCR0A = 0;
    TCNT0 = 255; //preload timer counter
    TIMSK0 = (1 << TOIE0); //enable overflow interrupt
    TCCR0B |= (1 << CS01) | (1 << CS00); //set prescaler to 64
    sei(); //enable global interrupts
}

```

```

//setup external interrupts
void InitExtInter(void){
    DDRD &= ~(1 << 2); //set PD2 as input
    PORTD |= (1 << 1) | (1<<2); //enable pull-up resistors
    EICRA = (1<<2); //trigger INT0 on falling edge
    EIMSK = (1<<INT0); //enable external interrupt 0
    sei(); //enable global interrupts
}

```

```

//initialize USART for serial communication
void USART_init(unsigned int ubrr){
    UBRROH = (unsigned char)(ubrr>>8);
    UBRROL = (unsigned char)ubrr;
    UCSR0B = (1 << TXEN0); //enable transmitter
    UCSR0C = (3 << UCSZ00); //set frame format: 8 data bits, no parity, 1 stop bit
}

```

```

//transmit string via USART
void USART_tx_string(char *data){
    while ((*data != '\0')){

```

```

        while (!(UCSR0A & (1 << UDRE0))); //wait for empty transmit buffer
        UDR0 = *data; //send character
        data++;
    }
}

int main(void){
    InitExtInter();
    InitTimer1();
    adc_init(); //initialize ADC
    init_IO();
    init_SPI();
    DDRB |= 0b00001110; //set PB3, PB1, PB2 as outputs for motor control
    DDRD |= (1<<ENABLE) | (1<<5) | (1<<6); //set pins as outputs
    OCR2A = 50; //set PWM duty cycle
    TCCR2A = (1<<COM2B1)|(1<<WGM21)|(1<<WGM20); //configure Timer2 for fast
PWM
    TCCR2B = 0x02; //set prescaler to 8

    while(1){
        read_adc();
        PORTD |= (1<<ENABLE); //enable motor
        PORTD |= (1<<5); //set MTR_1 high
        PORTD &= ~(1<<6); //set MTR_2 low

        OCR2B = adc_temp * 15; //adjust PWM based on ADC reading

        // Display handling in the SPI block
        if(alternator == 1){
            recTicks2 = recTicks;
            SHIFT_PORT &= ~LATCH; //pull LATCH low to start SPI transfer
            if(recTicks2/24 >= 10){
                spi_send((unsigned char)SEGMENT_MAP[1] - 0x80); //display
digit on the 7-segment
            }
            else{
                spi_send((unsigned char)SEGMENT_MAP[0] - 0x80);
            }
            spi_send((unsigned char)0xF2); //select digit
            SHIFT_PORT |= LATCH; //toggle latch to copy data to storage register
            SHIFT_PORT &= ~LATCH;
            alternator = 0;
        }
        else{
            spi_send((unsigned char)SEGMENT_MAP[(recTicks2/24) % 10]);
            spi_send((unsigned char)0xF4);
        }
    }
}

```

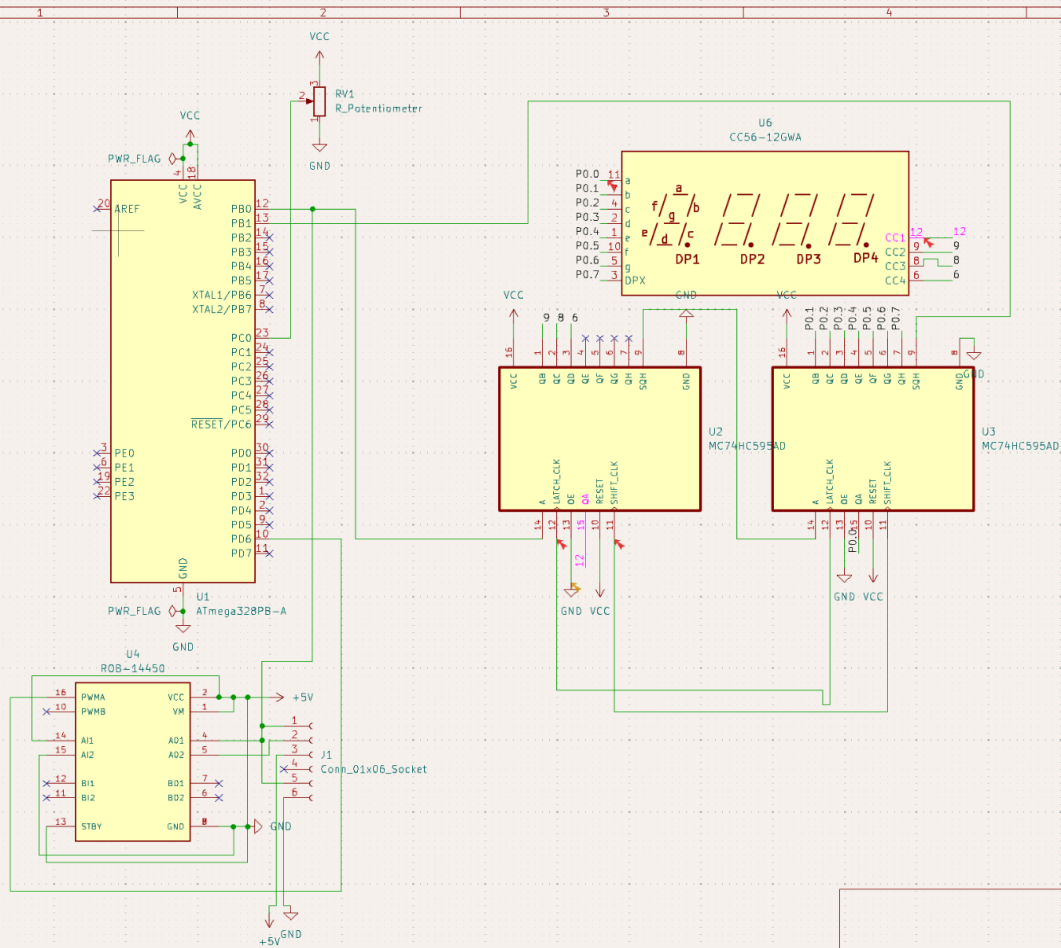
```
SHIFT_PORT |= LATCH;  
SHIFT_PORT &= ~LATCH;  
alternator = 1;
```

```
}
```

```
}
```

```
}
```

3. KICAD SCHEMATICS FOR TASK



Sheet: /
File: CPE301_DesignAssignment6_Schematics.kicad_sch

Title:

Size: A4
KiCad E.D.A. 8.0.0

Date:

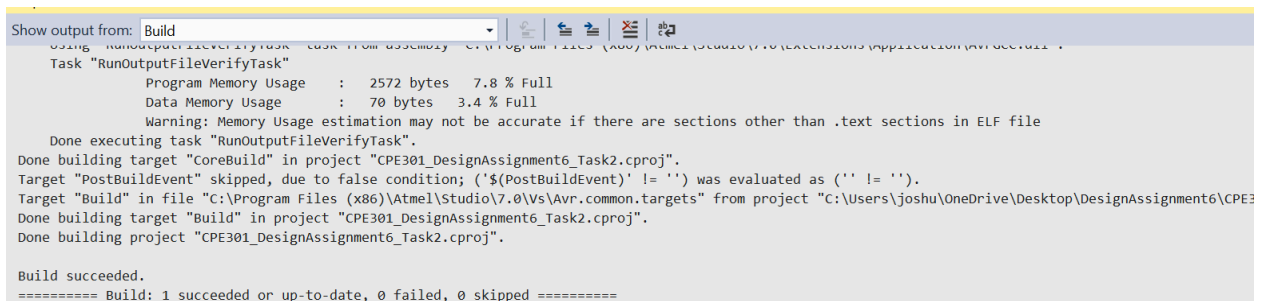
Rev:

Id: 1/1

4. SCREENSHOTS OF EACH TASK OUTPUT

```
Program Memory Usage : 552 bytes 2.2 % Full
Data Memory Usage : 46 bytes 2.2 % Full
Warning: Memory Usage estimation may not be accurate if there are sections other than .text sections in ELF file
Done executing task "RunOutputFileVerifyTask".
Done building target "CoreBuild" in project "CPE301_DesignAssignment6_Task3.cproj".
Target "PostBuildEvent" skipped, due to false condition; ('$(PostBuildEvent)' != '') was evaluated as ('' != '').
Target "Build" in file "C:\Program Files (x86)\Atmel\Studio\7.0\Vs\Avr.common.targets" from project "C:\Users\joshu\OneDrive\Desktop\DesignAssignment6\CPE301_DesignAssignment6_Task3.cproj".
Done building target "Build" in project "CPE301_DesignAssignment6_Task3.cproj".
Done building project "CPE301_DesignAssignment6_Task3.cproj".

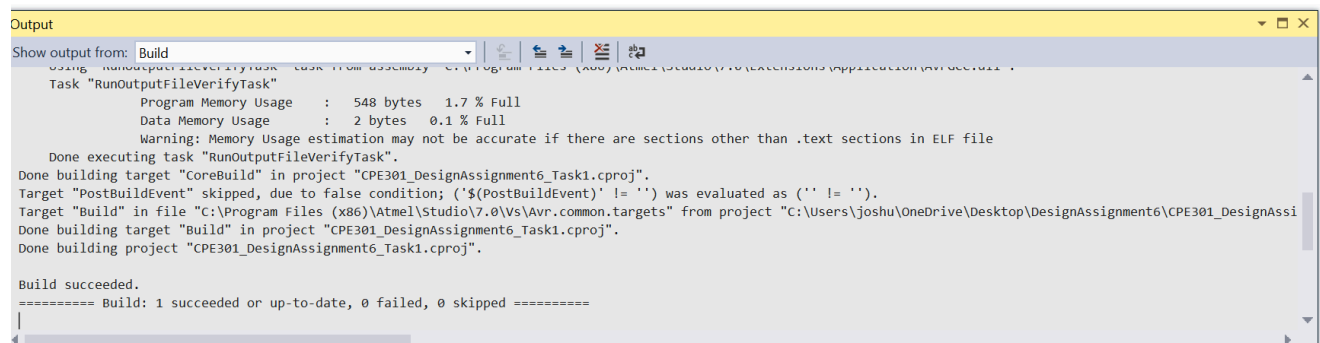
Build succeeded.
===== Build: 1 succeeded or up-to-date, 0 failed, 0 skipped =====
```



The screenshot shows the Visual Studio Output window with the dropdown menu set to 'Build'. The output text is as follows:

```
Using "RunOutputFileVerifyTask" task from assembly "C:\Program Files (x86)\Atmel\Studio\7.0\Extensions\application\avr\avrutil.dll".
Task "RunOutputFileVerifyTask"
    Program Memory Usage : 2572 bytes 7.8 % Full
    Data Memory Usage : 70 bytes 3.4 % Full
    Warning: Memory Usage estimation may not be accurate if there are sections other than .text sections in ELF file
    Done executing task "RunOutputFileVerifyTask".
Done building target "CoreBuild" in project "CPE301_DesignAssignment6_Task2.cproj".
Target "PostBuildEvent" skipped, due to false condition; ('$(PostBuildEvent)' != '') was evaluated as ('' != '').
Target "Build" in file "C:\Program Files (x86)\Atmel\Studio\7.0\Vs\Avr.common.targets" from project "C:\Users\joshu\OneDrive\Desktop\DesignAssignment6\CPE301_DesignAssignment6_Task2.cproj".
Done building target "Build" in project "CPE301_DesignAssignment6_Task2.cproj".
Done building project "CPE301_DesignAssignment6_Task2.cproj".

Build succeeded.
===== Build: 1 succeeded or up-to-date, 0 failed, 0 skipped =====
```



The screenshot shows the Visual Studio Output window with the dropdown menu set to 'Build'. The output text is as follows:

```
Using "RunOutputFileVerifyTask" task from assembly "C:\Program Files (x86)\Atmel\Studio\7.0\Extensions\application\avr\avrutil.dll".
Task "RunOutputFileVerifyTask"
    Program Memory Usage : 548 bytes 1.7 % Full
    Data Memory Usage : 2 bytes 0.1 % Full
    Warning: Memory Usage estimation may not be accurate if there are sections other than .text sections in ELF file
    Done executing task "RunOutputFileVerifyTask".
Done building target "CoreBuild" in project "CPE301_DesignAssignment6_Task1.cproj".
Target "PostBuildEvent" skipped, due to false condition; ('$(PostBuildEvent)' != '') was evaluated as ('' != '').
Target "Build" in file "C:\Program Files (x86)\Atmel\Studio\7.0\Vs\Avr.common.targets" from project "C:\Users\joshu\OneDrive\Desktop\DesignAssignment6\CPE301_DesignAssignment6_Task1.cproj".
Done building target "Build" in project "CPE301_DesignAssignment6_Task1.cproj".
Done building project "CPE301_DesignAssignment6_Task1.cproj".

Build succeeded.
===== Build: 1 succeeded or up-to-date, 0 failed, 0 skipped =====
```


5. SCREENSHOTS OF EACH DEMO (BOARD SETUP)

N/A: Read Note Above

6. VIDEO LINKS OF EACH DEMO

Task: https://www.youtube.com/playlist?list=PLZ09MA_uFt63me60r5XWtDUj2m21n4nUP

Student Academic Misconduct Policy

<http://studentconduct.unlv.edu/misconduct/policy.html>

"This assignment submission is my own, original work".

JOSHUA MARTINEZ